Theoretical Analysis of RAINBOWPLUS

Anonymous Authors
Under Review

1 Theoretical Analysis

In this section, we provide a formal theoretical analysis of RAINBOWPLUS, demonstrating that our approach addresses fundamental computational bottlenecks in quality-diversity search for adversarial prompt generation. We formalize the problem settings, establish complexity bounds, and prove that our multi-prompt fitness evaluation achieves significant asymptotic improvements over pairwise comparison methods.

1.1 Problem Formalization

We begin by formally defining three variants of the quality-diversity search problem for adversarial prompt generation, progressing from the baseline Rainbow framework to our proposed RAINBOWPLUS.

Definition 1.1 (Standard Rainbow Problem)**.** In the standard Rainbow framework, the archive maintains a single prompt per cell. For each cell indexed by descriptor $z \in \mathcal{Z}$, we have:

- Archive cell: $G[z] = x$ (single prompt)
- Response storage: $R[z] = r$ (single response)
- Update mechanism: A pairwise preference function $p : \mathcal{X} \times \mathcal{X} \to \mathcal{X}$ determines whether a candidate prompt x' replaces the incumbent x, where $p(x, x') = x'$ if x' is preferred
- Generation: One candidate prompt per iteration
- Update frequency: At most one prompt updated per iteration

Definition 1.2 (Multi-Prompt Rainbow Problem)**.** A natural extension of Rainbow to store multiple prompts per cell while retaining the pairwise comparison mechanism. For each cell $z \in \mathcal{Z}$:

- Archive cell: $G[z] = \{x_1, x_2, \ldots, x_m\}$ (set of m prompts)
- Response storage: $R[z] = \{r_1, r_2, \ldots, r_m\}$ (corresponding responses)
- Update mechanism: Pairwise preference function $p(x, x')$ compares candidate x' against all existing prompts in $G[z]$. The candidate is added if $p(x', x) = x'$ for all $x \in G[z]$

- Generation: One candidate prompt per iteration
- Update frequency: At most one prompt added per iteration

Definition 1.3 (RAINBOWPLUS Problem)**.** Our proposed framework that employs a multi-element archive with a probabilistic fitness function for efficient batch evaluation. For each cell $z \in \mathcal{Z}$:

- Archive cell: $G[z] = \{x_1, x_2, \ldots, x_m\}$ (set of m prompts)
- Response storage: $R[z] = \{r_1, r_2, \ldots, r_m\}$ (corresponding responses)
- Fitness storage: $F[z] = \{f(x_1), f(x_2), \ldots, f(x_m)\}$ where $f : \mathcal{X} \to [0, 1]$ is a probabilistic fitness function
- Update mechanism: Threshold-based fitness function $f(x') > \eta$ determines acceptance, where $\eta \in [0, 1]$ is a fitness threshold
- Generation: M candidate prompts per iteration (batch generation)
- Update frequency: Multiple prompts (up to M) added per iteration

Remark 1.4. The key distinctions are: (1) Multi-Prompt Rainbow generates one candidate per iteration and uses sequential pairwise comparisons, while RAINBOWPLUS generates M candidates per iteration and uses independent fitness evaluations; (2) Multi-Prompt Rainbow requires a candidate to dominate all existing prompts in the cell, while RAINBOWPLUS uses an absolute fitness threshold η, enabling multiple prompts to be added simultaneously.

1.2 Theoretical Assumptions

To establish rigorous complexity bounds, we introduce the following assumption that characterizes an idealized update scenario.

Assumption 1.5 (Perfect Update Condition)**.** Suppose the mutation operator and diversity filtering mechanism consistently generate prompts that satisfy the following conditions:

1. **Diversity condition:** All generated candidate prompts satisfy the similarity threshold constraint $\text{sim}(x, x') < \theta$ for all $x \in G[z]$.
2. **Quality condition:**
   - For Multi-Prompt Rainbow: The generated candidate x' is sufficiently effective that $p(x', x) = x'$ for all $x \in G[z]$, ensuring successful addition to the archive
   - For RAINBOWPLUS: All M generated candidates satisfy $f(x'_i) > \eta$ for $i = 1, \ldots, M$
3. **Initialization condition:** The archive is initialized with one seed prompt per cell from a dataset D, so each cell starts with $|G[z]| = 1$
4. **Cell selection:** In each iteration, we select a cell with the current minimum number of prompts to ensure uniform filling across the archive

Under these conditions, after N iterations, the archive contains larger than 1 prompts per cell, where $N = |\mathcal{Z}|$ is the number of cells.

Remark 1.6. Assumption 1.5 provides an idealized setting for analyzing the best-case complexity. This assumption enables us to isolate and analyze the fundamental complexity differences between the pairwise comparison and fitness-based approaches.

1.3 Complexity Analysis

We now formally establish the computational complexity of each approach, demonstrating that RAINBOWPLUS achieves a significant asymptotic improvement. We measure complexity in terms of the number of LLM comparison or evaluation operations, which dominate the computational cost in red-teaming scenarios.

Lemma 1.7 (Multi-Prompt Rainbow Update Complexity)**.** *For Multi-Prompt Rainbow (Definition 1.2), when a cell $G[z]$ contains m prompts $\{x_1, x_2, \ldots, x_m\}$, a candidate prompt x' is added to the archive if and only if $p(x', x_i) = x'$ for all $x_i \in G[z]$. This verification requires m pairwise comparisons.*

Proof. To determine whether x' should be added to $G[z]$, the algorithm must verify that x' is preferred over every existing prompt in the cell. This requires computing:

$$p(x', x_1),\ p(x', x_2),\ \ldots,\ p(x', x_m) \tag{1}$$

Each comparison $p(x', x_i)$ involves querying the Judge LLM π_J to compare the responses $\pi_T(x')$ and $\pi_T(x_i)$, which constitutes one computational operation. Since there are m prompts in $G[z]$, exactly m comparisons are required. The prompt x' is added to the archive if and only if all m comparisons favor x'. □

Lemma 1.8 (RAINBOWPLUS Update Complexity)**.** *For* RAINBOWPLUS *(Definition 1.3), when evaluating M candidate prompts $\{x'_1, x'_2, \ldots, x'_M\}$ for a cell $G[z]$, each prompt x'_j is added to the archive if and only if $f(x'_j) > \eta$. This verification requires M independent fitness evaluations.*

Proof. The fitness function $f : \mathcal{X} \to [0, 1]$ evaluates each candidate prompt independently by computing:

$$f(x'_j) = \mathbb{P}(\pi_J(\pi_T(x'_j)) = \text{“unsafe”}) \tag{2}$$

where π_T is the Target LLM and π_J is the Judge LLM. For M candidate prompts, we compute:

$$\{f(x'_1), f(x'_2), \ldots, f(x'_M)\} \tag{3}$$

Each evaluation $f(x'_j)$ requires one forward pass through π_T to generate a response and one forward pass through π_J to classify it, constituting one computational operation. Since fitness evaluations are independent of the current contents of $G[z]$ and of each other, the total number of evaluations is exactly M, regardless of $|G[z]|$. □

Theorem 1.9 (Multi-Prompt Rainbow Time Complexity)**.** *Under Assumption 1.5, Multi-Prompt Rainbow requires $\Theta(M^2 N)$ comparison operations to grow an archive of N cells from 1 prompt per cell to $M + 1$ prompts per cell.*

Proof. We analyze the total number of pairwise comparisons required across all iterations using a phase-based argument. Since Multi-Prompt Rainbow generates one candidate per iteration and we select cells with minimum prompts (Assumption 1.5), the filling process proceeds in phases where all cells have the same number of prompts.

Initial State. Each of the N cells is initialized with 1 seed prompt (Assumption 1.5, initialization condition).

Phase Structure. The filling process consists of M phases, where phase j adds the $(j+1)$-th prompt to each cell:

- **Phase 1** (iterations 1 to N): Each selected cell contains 1 prompt. By Lemma 1.7, adding candidate x' requires 1 comparison. After this phase, each cell has 2 prompts.
- **Phase 2** (iterations $N+1$ to $2N$): Each selected cell contains 2 prompts. By Lemma 1.7, adding candidate x' requires 2 comparisons. After this phase, each cell has 3 prompts.
- $\vdots$
- **Phase** j (iterations $(j-1)N+1$ to jN): Each selected cell contains j prompts. By Lemma 1.7, adding candidate x' requires j comparisons. After this phase, each cell has $j+1$ prompts.
- $\vdots$
- **Phase** M (iterations $(M-1)N+1$ to MN): Each selected cell contains M prompts. By Lemma 1.7, adding candidate x' requires M comparisons. After this phase, each cell has $M+1$ prompts.

Total Comparisons. In phase j, there are N iterations (one per cell), and each iteration requires j comparisons. The total number of comparisons (operations) across all M phases is:

$$O_{\text{total}} = \sum_{j=1}^{M} \underbrace{N \cdot j}_{\text{Phase } j \text{ comparisons}} \tag{4}$$

$$= N \sum_{j=1}^{M} j \tag{5}$$

$$= N \cdot \frac{M(M+1)}{2} \tag{6}$$

$$= \frac{M^2 N + MN}{2} \tag{7}$$

$$= \Theta(M^2 N) \tag{8}$$

Therefore, the time complexity of Multi-Prompt Rainbow is $\Theta(M^2 N)$, which scales quadratically with the target archive capacity M. □

Theorem 1.10 (RAINBOWPLUS Time Complexity)**.** *Under Assumption 1.5,* RAINBOWPLUS *requires* $\Theta(MN)$ *fitness evaluations to grow an archive of* N *cells from* 1 *prompt per cell to* $M + 1$ *prompts per cell.*

Proof. In RAINBOWPLUS, the archive is filled by generating and evaluating M candidate prompts per iteration (Definition 1.3). We select cells with the minimum number of prompts to ensure uniform filling (Assumption 1.5).

Initial State. Each of the N cells is initialized with 1 seed prompt.

Iteration Structure. Since we select cells with minimum prompts and add M prompts per iteration:

- **Iteration 1:** Select a cell with 1 prompt. Generate M candidates using the Mutator LLM π_M, which requires M generation operations. By Lemma 1.8, evaluating these candidates requires M fitness evaluations. All M candidates satisfy $f(x'_i) > \eta$ and are added (Assumption 1.5). The selected cell now has $M + 1$ prompts.
- **Iteration 2:** Select another cell with 1 prompt (minimum among remaining cells). Generate M candidates (M generation operations) and evaluate them (M fitness evaluations). Add all M candidates. This cell now has $M + 1$ prompts.
- $\vdots$
- **Iteration** k (for $k = 1, 2, \ldots, N$): Select the k-th cell with 1 prompt. Generate M candidates (M generation operations) and evaluate them (M fitness evaluations). Add all M candidates. This cell now has $M + 1$ prompts.
- $\vdots$
- **Iteration** N**:** Select the final cell with 1 prompt. Generate M candidates (M generation operations) and evaluate them (M fitness evaluations). Add all M candidates. This cell now has $M + 1$ prompts.

Operations per Iteration. Each iteration consists of two independent phases:

1. **Generation Phase:** The Mutator LLM π_M generates M candidate prompts, requiring M generation operations. This phase is independent of the evaluation phase and can be performed separately.
2. **Evaluation Phase:** The Judge LLM π_J evaluates the fitness of all M candidates, requiring M fitness evaluation operations (Lemma 1.8).

Since both phases scale linearly with M and are performed sequentially (generation followed by evaluation), the total cost per iteration is $\mathcal{O}(M) + \mathcal{O}(M) = \mathcal{O}(M)$.

Total Operations. After N iterations, all cells have been updated exactly once, growing from 1 prompt to $M + 1$ prompts. The total number of operations is:

$$O_{\text{total}} = \sum_{k=1}^{N} [\text{Generation cost} + \text{Evaluation cost}] \tag{9}$$

$$= \sum_{k=1}^{N} (M + M) \tag{10}$$

$$= \sum_{k=1}^{N} 2M \tag{11}$$

$$= 2MN \tag{12}$$

$$= \Theta(MN) \tag{13}$$

Therefore, the time complexity of RAINBOWPLUS is $\Theta(MN)$, which scales linearly with the target archive capacity M. This represents a significant asymptotic improvement over Multi-Prompt Rainbow's $\Theta(M^2N)$ complexity. □

Corollary 1.11 (Complexity Reduction)**.** RAINBOWPLUS *achieves a speedup factor of* $\Theta(M)$ *over Multi-Prompt Rainbow for target archive capacity* M.

Proof. The speedup factor is:

$$\text{Speedup} = \frac{\Theta(M^2N)}{\Theta(MN)} = \Theta(M) \tag{14}$$

□